

# CloudDocs Architecture & Setup Reference

Full-Stack Guide: React + TypeScript, .NET 8/9, PostgreSQL, and AWS S3

.NET 8 / 9

REACT & TS

POSTGRES SQL

AWS S3

EF CORE

## 1. System Architecture & Workflow

This guide details the complete configuration for building a secure, scalable document management application. The system utilizes direct-to-S3 browser uploads via AWS Presigned URLs, persisting metadata in PostgreSQL through EF Core.

| Component       | Tech Stack              | Role & Responsibility                                                                   |
|-----------------|-------------------------|-----------------------------------------------------------------------------------------|
| Frontend Client | React, TypeScript, Vite | Renders UI, requests presigned URLs, and directly uploads files to S3.                  |
| REST API        | .NET 8 / 9 Web API      | Generates presigned upload URLs, authenticates requests, manages metadata.              |
| Database        | PostgreSQL + EF Core    | Stores document metadata (file key, size, type, timestamps) with connection resilience. |
| Object Storage  | AWS S3                  | Stores physical binary file content securely with lifecycle rules.                      |

## 2. NuGet Package Installation

Execute the following commands in your .NET project directory to install the required data access and AWS SDK dependencies:

```
# Install PostgreSQL provider for Entity Framework Core
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL

# Install AWS S3 SDK for file operations
dotnet add package AWSSDK.S3

# Install AWS Dependency Injection Extension for .NET Core
dotnet add package AWSSDK.Extensions.NETCore.Setup
```

## 3. Application Configuration (appsettings.json)

Configure database connection strings, AWS region specifications, and CORS origins in your root configuration file:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "Default": "Host=postgres.amazonaws.com;Database=cloud_docs;User=postgres;Password=postgres;Port=5432"
  },
  "AWS": {
    "Region": "us-east-1"
  },
  "Cors": {
    "Origins": "https://cloud-docs.com"
  }
}
```

```

    "AllowedHosts": "*",
    "ConnectionStrings": {
        "DefaultConnection":
"Host=localhost;Port=5432;Database=clouddocs_db;Username=postgres;Password=your_password_here"
    },
    "AWS": {
        "Region": "us-east-1",
        "S3": {
            "BucketName": "your-app-upload-bucket-name"
        }
    },
    "Cors": {
        "AllowedOrigins": [
            "http://localhost:5173",
            "http://localhost:3000"
        ]
    }
}

```

 **AWS Security Best Practice:** Never hardcode AccessKey or SecretKey inside appsettings.json. The AWS SDK automatically resolves credentials from standard environment variables (AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS\_KEY), standard credentials file (~/.aws/credentials), or IAM Roles in deployment environments (ECS / EC2 / App Runner).

## 4. Application Startup & Dependency Injection (Program.cs)

This file configures PostgreSQL resilience, registers AWS S3 client services, establishes CORS policy, and configures the middleware pipeline.

```

using Amazon.S3;
using CloudDocs.API.Data;
using CloudDocs.API.Services;
using CloudDocs.API.Services.Interfaces;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// =====
// 1. Controller & OpenAPI Setup
// =====
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// =====
// 2. Configure PostgreSQL with EF Core
// =====
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
?? throw new InvalidOperationException("Database connection string 'DefaultConnection' not found.");

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(connectionString, npgsqlOptions =>
    {
        // Enable automatic retry on transient failures
        npgsqlOptions.EnableRetryOnFailure(
            maxRetryCount: 5,

```

```

        maxRetryDelay: TimeSpan.FromSeconds(10),
        errorCodesToAdd: null);
    }));

// =====
// 3. Configure AWS S3 Client & Custom Services
// =====
// Registers IAmazonS3 into DI container reading from "AWS" section in appsettings.json
builder.Services.AddAWSService<IAmazonS3>(builder.Configuration.GetAWSOptions("AWS"));

// Register custom Storage Service abstraction
builder.Services.AddScoped<IStorageService, StorageService>();

// =====
// 4. Configure CORS for React Frontend
// =====
var allowedOrigins = builder.Configuration.GetSection("Cors:AllowedOrigins").Get<string[]>()
    ?? new[] { "http://localhost:5173" };

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins(allowedOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

// =====
// 5. Middleware Pipeline Configuration
// =====
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "CloudDocs API v1");
    });
}

app.UseHttpsRedirection();
app.UseCors("AllowFrontend");
app.UseAuthorization();
app.MapControllers();

// Automatically apply database migrations on startup in development
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}

app.Run();

```

## 5. S3 Storage Service Implementation (StorageService.cs)

The storage service consumes the injected `IAmazonS3` instance to generate pre-signed upload URLs for direct browser-to-S3 binary transfers.

```
using Amazon.S3;
using Amazon.S3.Model;
using CloudDocs.API.Services.Interfaces;

namespace CloudDocs.API.Services;

public class StorageService : IStorageService
{
    private readonly IAmazonS3 _s3Client;
    private readonly string _bucketName;

    public StorageService(IAmazonS3 s3Client, IConfiguration config)
    {
        _s3Client = s3Client;
        _bucketName = config["AWS:S3:BucketName"]
            ?? throw new ArgumentNullException("AWS:S3:BucketName configuration is missing.");
    }

    public string GeneratePresignedUploadUrl(string fileKey, string contentType)
    {
        var request = new GetPreSignedUrlRequest
        {
            BucketName = _bucketName,
            Key = fileKey,
            Verb = HttpVerb.PUT,
            Expires = DateTime.UtcNow.AddMinutes(15),
            ContentType = contentType
        };

        return _s3Client.GetPreSignedURL(request);
    }
}
```

## 6. Required AWS S3 Bucket CORS Configuration

To enable direct uploads from your React web application to AWS S3, you must configure the Cross-Origin Resource Sharing (CORS) rules on your S3 bucket via the AWS Console or CLI:

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["PUT", "POST", "GET", "HEAD"],
    "AllowedOrigins": ["http://localhost:5173", "http://localhost:3000"],
    "ExposeHeaders": ["ETag"]
  }
]
```



### Next Steps for Implementation:

- Run `dotnet ef migrations add InitialCreate` to create database migrations.
- Execute `dotnet run` to start the backend API and verify Swagger documentation at `/swagger`.

- Connect your React frontend using Axios or Fetch to request presigned URLs from `/api/documents/presigned-url`.